```
# KLUE-BERT 네이버 영화 리뷰 감성 분류
## 상세 버전 (구조 이해용)
#CLS 토큰을 직접 꺼내서 분류층에 연결하는 방식
```

```
!pip install transformers==4.40.0
```

숨겨진 출력 표시

```
# 라이브러리
import pandas as pd
import numpy as np
import urllib.request
import os
from tqdm import tqdm
import tensorflow as tf
from transformers import BertTokenizer, TFBertModel
```

```
# 데이터 로드

urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/e9t/nsmc/master/ratings_train.txt",
    filename="ratings_train.txt"
)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/e9t/nsmc/master/ratings_test.txt",
    filename="ratings_test.txt"
)

train_data = pd.read_table('ratings_train.txt')
test_data = pd.read_table('ratings_test.txt')

print('훈련용 리뷰 개수:', len(train_data))
print('테스트용 리뷰 개수:', len(test_data))
```

```
훈련용 리뷰 개수: 150000
테스트용 리뷰 개수: 50000
```

```
# 전처리

train_data.drop_duplicates(subset=['document'], inplace=True)
train_data = train_data.dropna(how='any')
print('훈련 데이터의 리뷰 수:', len(train_data))

test_data = test_data.dropna(how='any')
print('테스트 데이터의 리뷰 수:', len(test_data))
```

```
훈련 데이터의 리뷰 수: 146182
테스트 데이터의 리뷰 수: 49997
```

```
# 토크나이저 확인하기
```

```
tokenizer = BertTokenizer.from_pretrained('klue/bert-base')

print(tokenizer.tokenize('보는 내내 그대로 들어맞는 예측 카리스마 없는 악역'))
print(tokenizer.encode('보는 내내 그대로 들어맞는 예측 카리스마 없는 악역'))
print(tokenizer.decode(tokenizer.encode('보는 내내 그대로 들어맞는 예측 카리스마 없는 악역')))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If yo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%    289/289 [00:00<00:00, 27.5kB/s]
vocab.txt:    248k/? [00:00<00:00, 7.02MB/s]
special_tokens_map.json: 100%    125/125 [00:00<00:00, 18.5kB/s]
tokenizer.json:    495k/? [00:00<00:00, 30.0MB/s]
config.json: 100%    425/425 [00:00<00:00, 53.9kB/s]
['보', '##는', '내내', '그대로', '들어맞', '##는', '예측', '카리스마', '없', '##는', '악역']
[2, 1160, 2259, 6404, 4311, 20657, 2259, 5501, 13132, 1415, 2259, 23713, 3]
[CLS] 보는 내내 그대로 들어맞는 예측 카리스마 없는 악역 [SEP]
```

```
# 특수 토큰 확인
print(tokenizer.cls_token, ':', tokenizer.cls_token_id)  # [CLS] : 2
print(tokenizer.sep_token, ':', tokenizer.sep_token_id)  # [SEP] : 3
print(tokenizer.pad_token, ':', tokenizer.pad_token_id)  # [PAD] : 0
```

```
[CLS] : 2
[SEP] : 3
[PAD] : 0
```

```
# 패딩 적용 예시
max_seq_len = 128
encoded_result = tokenizer.encode(
    '전율을 일으키는 영화. 다시 보고 싶은 영화',
    max_length=max_seq_len,
    padding='max_length',
    truncation=True
)
print(encoded_result)
print('길이:', len(encoded_result))

# 세그먼트 인코딩 (단일 문장이면 전부 0)
print([0] * max_seq_len)

# 어텐션 마스크
valid_num = len(tokenizer.encode('전율을 일으키는 영화. 다시 보고 싶은 영화'))
print(valid_num * [1] + (max_seq_len - valid_num) * [0])
```

```
[2, 1537, 2534, 2069, 6572, 2259, 3771, 18, 3690, 4530, 1335, 2073, 3771, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
길이: 128
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

```
# 전체 데이터 인코딩 함수

def convert_examples_to_features(examples, labels, max_seq_len, tokenizer):
    input_ids, attention_masks, token_type_ids, data_labels = [], [], [], []

    for example, label in tqdm(zip(examples, labels), total=len(examples)):
        input_id = tokenizer.encode(
            example,
            max_length=max_seq_len,
            padding='max_length',
            truncation=True
        )
        padding_count = input_id.count(tokenizer.pad_token_id)
        attention_mask = [1] * (max_seq_len - padding_count) + [0] * padding_count
        token_type_id = [0] * max_seq_len

        assert len(input_id) == max_seq_len
        assert len(attention_mask) == max_seq_len
        assert len(token_type_id) == max_seq_len

        input_ids.append(input_id)
        attention_masks.append(attention_mask)
        token_type_ids.append(token_type_id)
        data_labels.append(label)

    input_ids = np.array(input_ids, dtype=int)
    attention_masks = np.array(attention_masks, dtype=int)
    token_type_ids = np.array(token_type_ids, dtype=int)
    data_labels = np.asarray(data_labels, dtype=np.int32)

    return (input_ids, attention_masks, token_type_ids), data_labels
    train_X, train_y = convert_examples_to_features(
    train_data['document'], train_data['label'],
    max_seq_len=max_seq_len, tokenizer=tokenizer
)
test_X, test_y = convert_examples_to_features(
    test_data['document'], test_data['label'],
    max_seq_len=max_seq_len, tokenizer=tokenizer
)
```

```
100%|██████████| 49997/49997 [00:09<00:00, 5027.75it/s]
```

```
train_X, train_y = convert_examples_to_features(
    train_data['document'], train_data['label'],
    max_seq_len=max_seq_len, tokenizer=tokenizer
)
test_X, test_y = convert_examples_to_features(
    test_data['document'], test_data['label'],
    max_seq_len=max_seq_len, tokenizer=tokenizer
)
```

```
100%|██████████| 146182/146182 [00:27<00:00, 5300.90it/s]
100%|██████████| 49997/49997 [00:09<00:00, 5453.34it/s]
```

```
# 인코딩 결과 확인
print('단어에 대한 정수 인코딩:', train_X[0][0])
print('어텐션 마스크:', train_X[1][0])
print('세그먼트 인코딩:', train_X[2][0])
print('각 인코딩의 길이:', len(train_X[0][0]))
print('정수 인코딩 복원:', tokenizer.decode(train_X[0][0]))
print('레이블:', train_y[0])
```

```
단어에 대한 정수 인코딩: [   2 1376  831 2604   18   18 4229 9801 2075 2203 2182 4243    3    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0]
어텐션 마스크: [1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
세그먼트 인코딩: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
각 인코딩의 길이: 128
정수 인코딩 복원: [CLS] 아 더빙.. 진짜 짜증나네요 목소리 [SEP] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
레이블: 0
```

```
## 5. 커스텀 모델 클래스 정의
# BERT 위에 분류층을 직접 올리는 구조

class CustomBertClassifier(tf.keras.Model):
    def __init__(self, model_name):
        super(CustomBertClassifier, self).__init__()
        # BERT 모델 로드 (from_pt=True: PyTorch 가중치를 TF로 변환)
        self.bert = TFBertModel.from_pretrained(model_name, from_pt=True)
        # 분류층: 출력 1개 (긍정 확률), sigmoid 활성화
        self.classifier = tf.keras.layers.Dense(
            1,
            kernel_initializer=tf.keras.initializers.TruncatedNormal(0.02),
            activation='sigmoid',
```

```
            name='classifier'
        )

    def call(self, inputs):
        input_ids, attention_mask, token_type_ids = inputs
        outputs = self.bert(
            input_ids=input_ids,
            attention_mask=attention_mask,
            token_type_ids=token_type_ids
        )
        # outputs[1]: CLS 토큰 위치의 벡터 (shape: batch_size x 768)
        cls_token = outputs[1]
        prediction = self.classifier(cls_token)
        return prediction
```

```
# 모델 학습

model = CustomBertClassifier('klue/bert-base')
optimizer = tf.keras.optimizers.Adam(learning_rate=5e-5)
loss = tf.keras.losses.BinaryCrossentropy()
model.compile(optimizer=optimizer, loss=loss, metrics=['accuracy'])

model.fit(train_X, train_y, epochs=2, batch_size=64, validation_split=0.2)

results = model.evaluate(test_X, test_y, batch_size=1024)
print('test loss, test acc:', results)
```

```
pytorch_model.bin: 100%                                   445M/445M [00:02<00:00, 2.02GB/s]
Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFBertModel: ['cls.predictions.transform.LayerNorm.bias', 'cls.predictions.transform.dense.weight', 'cls.predictions.decode
- This IS expected if you are initializing TFBertModel from a PyTorch model trained on another task or with another architecture (e.g. initializing a TFBertForSequenceClassification model from a BertForPreT
- This IS NOT expected if you are initializing TFBertModel from a PyTorch model that you expect to be exactly identical (e.g. initializing a TFBertForSequenceClassification model from a BertForSequenceClass
All the weights of TFBertModel were initialized from the PyTorch model.
If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions without further training.
Epoch 1/2
1828/1828 [==============================] - 3172s 2s/step - loss: 0.2819 - accuracy: 0.8800 - val_loss: 0.2397 - val_accuracy: 0.9016
Epoch 2/2
1828/1828 [==============================] - 3125s 2s/step - loss: 0.1873 - accuracy: 0.9271 - val_loss: 0.2377 - val_accuracy: 0.9045
49/49 [==============================] - 441s 9s/step - loss: 0.2462 - accuracy: 0.9001
test loss, test acc: [0.24624715745449066, 0.9000539779663086]
```

```
# 예측 함수
def sentiment_predict(new_sentence):
    input_id = tokenizer.encode(
        new_sentence,
        max_length=max_seq_len,
        padding='max_length',
        truncation=True
    )
    padding_count = input_id.count(tokenizer.pad_token_id)
    attention_mask = [1] * (max_seq_len - padding_count) + [0] * padding_count
    token_type_id = [0] * max_seq_len
```

```
    input_ids = np.array([input_id])
    attention_masks = np.array([attention_mask])
    token_type_ids = np.array([token_type_id])

    score = model.predict([input_ids, attention_masks, token_type_ids])[0][0]

    if score > 0.5:
        print("{:.2f}% 확률로 긍정 리뷰입니다.₩n".format(score * 100))
    else:
        print("{:.2f}% 확률로 부정 리뷰입니다.₩n".format((1 - score) * 100))

sentiment_predict('이 영화 존잼입니다 대박')
sentiment_predict('이 영화 핵노잼 ㅠㅠ')
sentiment_predict('이딴게 영화냐 ㅉㅉ')
sentiment_predict('와 개쩐다 정말 세계관 최강자들의 영화다')
```

```
1/1 [==============================] - 3s 3s/step
98.19% 확률로 긍정 리뷰입니다.

1/1 [==============================] - 0s 58ms/step
98.57% 확률로 부정 리뷰입니다.

1/1 [==============================] - 0s 61ms/step
96.58% 확률로 부정 리뷰입니다.

1/1 [==============================] - 0s 61ms/step
92.10% 확률로 긍정 리뷰입니다.
```